



Classification of application reviews into software maintenance tasks using data mining techniques

Assem Al-Hawari¹ · Hassan Najadat¹ · Raed Shatnawi¹ 

Published online: 28 August 2020

© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract

Mobile application reviews are considered a rich source of information for software engineers to provide a general understanding of user requirements and technical feedback to avoid main programming issues. Previous researches have used traditional data mining techniques to classify user reviews into several software maintenance tasks. In this paper, we aim to use associative classification (AC) algorithms to investigate the performance of different classifiers to classify reviews into several software maintenance tasks. Also, we proposed a new AC approach for review mining (ACRM). Review classification needs preprocessing steps to apply natural language preprocessing and text analysis. Also, we studied the influence of two feature selection techniques (information gain and chi-square) on classifiers. Association rules give a better understanding of users' intent since they discover the hidden patterns in words and features that are related to one of the maintenance tasks, and present it as class association rules (CARs). For testing the classifiers, we used two datasets that classify reviews into four different maintenance tasks. Results show that the highest accuracy was achieved by AC algorithms for both datasets. ACRM has the highest precision, recall, F-score, and accuracy. Feature selection helps improving the classifiers' performance significantly.

Keywords Associative classification · Software reviews mining · Interesting measures

✉ Raed Shatnawi
raedamin@just.edu.jo

Assem Al-Hawari
aralhawari15@cit.just.edu.jo

Hassan Najadat
najadat@just.edu.jo

¹ Jordan University of Science and Technology, Irbid, Jordan

1 Introduction

User feedback and rating are very important for both users and developers and represent a rich source of information. Users can rate an app from one to five stars and write a review about their experience or problems faced during downloading, upgrading, or running the app. Moreover, users can ask developers to enhance or add features. As we can see, these feedbacks are very important for developers in respect to software engineering (SE) maintenance tasks to improve their apps. We have a huge amount of user feedback as text reviews. Users add reviews every day, which makes it very difficult to track all of these reviews. This is a challenge faced by developers; some apps have tens of thousands of reviews. These reviews are unstructured data and may contain numbers, symbols, or even informal language. Since these reviews are written by the users, they have the freedom to write them in their way.

In addition, some reviews have only a few specific words that indicate user intention. Furthermore, many reviews have no useful information for developers, for example: “I hate this app” or “this is a great app, love it.” These reviews do not provide useful information for developers. Other reviews like “This is an ok app. I still love it. Not amazing but not terrible” gives a rating by words; even it has a rating from one to five stars as well. The emotional sentiment in the reviews has a weak correlation with the numerical score (Martens and Johann 2017). For instance, a user may rate an application as five stars while another user may write the same review and rate the app with three stars.

To get the benefits of user reviews, data mining techniques are used to extract the knowledge that the developers are seeking. However, reviews are shorter than any other text or document, so it needs special mining techniques such as classification and association rule mining. Applying text classification or categorization to documents is easier than review classification because documents contain more words related to a specific category, and that makes the classification process easier. Reviews have few words, which make mining the intention of the user harder. This paper aims to mine user reviews to extract knowledge that can be used by developers in identifying maintenance tasks.

Classification algorithms and association rules mining techniques were used to mine app reviews to extract different maintenance tasks; Martens et al. have classified reviews to four maintenance tasks: bug reports, feature request, user experience, and rating (Martens and Johann 2017). While Guzman et al. have proposed seven maintenance tasks: bug report, feature strength, feature shortcoming, user request, praise, complaint, and usage scenario (Guzman and El-Haliby 2015). In this paper, we evaluate classification approaches on two datasets, each one of them has reviews belonging to four main maintenance tasks. The first dataset has the following categories: bug reports or problem discovery, feature requests, information giving, and information seeking (Panichella et al. n.d.). The second dataset has the following categories: problem discovery, feature request, rating, and user experience, which includes information that can help developers to maintain their software (Maalej et al. 2016). Many researchers have implemented a text classification using word vectors that are extracted from reviews as features, and they used the natural language processing to build a structured dataset as a first step, then they applied some traditional classifiers to mine the reviews.

In this paper, we use the same traditional classifiers that were used in previous studies (such as NB, J48, SVM, GBT, and random forest (RF)), and we propose to use associative classification (AC) algorithms (such as CBA, CBA2, CMAR, CPAR). We also experiment with a new approach (ACRM) as an associative classification algorithm. ACRM depends on

two main interesting measures (IMs), confidence and conviction, to extract the class association rules (CARs).

The main problem addressed in this work is the user reviews' classification. Reviews are a short text, which contains few words. The frequency of words that belongs to the same type of reviews is usually low, and that makes it challenging to achieve high accuracy with multiclass classification. Datasets should be prepared by applying preprocessing procedures based on NLP and text.

The main contribution is to find the best classifiers to classify user reviews into software maintenance tasks. Other contributions include:

- Evaluate new classifiers that were not used before to classify reviews into maintenance tasks such as the AC algorithms.
- Proposing the ACRM approach as a competitive AC algorithm.
- Discovering the influence of feature selection with several classifiers concerning classification accuracy.
- Extracting CARs that help the developers with a better understanding of users' intent.

The rest of the paper is organized as follows: Section 2 shows an overview of data mining techniques, data mining in software engineering, and text classification. Section 3 summarizes the major tenets of the relevant literature in the field. Section 4 highlights the dataset preparation and presents the newly proposed ACRM approach in detail. Section 5 presents the experiments as well as the results and discussion. Section 6 discusses the threats to validity for the experiments under study. Finally, the conclusion of the study is discussed in Section 7.

2 Background

This section presents an overview of the used data mining techniques.

2.1 Association rules

Association rules were introduced in (Agrawal et al. 1993) to discover the hidden patterns in the transaction of market data, where one transaction is a set of items. Association rules are focused on extracting the frequent pattern from data itemsets. The extracted rules from these frequent itemsets depend on two thresholds factors, minimum support (minsup) and minimum confidence (minconf). Two famous algorithms are used to extract frequent itemsets, the Apriori (Agrawal and Srikant 1994) and the FP-Growth algorithm (Han et al. 2000).

2.2 Classification

Classification aims to find the category or the class of a new instance of data by building a model of a classifier, which predicts the class label of that instance. A classifier is built using a training dataset, where it reads all the observations from training instances that are already classified to build a model and then apply them to any new coming instance (Umadevi and Marseline 2017). Each algorithm has its way to read the training instances and build the classifier. The learning process could be either eager or lazy learning, where lazy learning stores training and waits until it is given a test instance, so it takes less time in the training

phase and more time in classification or predicting process. The eager learning builds a classifier from training data then applies the model to a new instance for classification, and that causes more time in training and less time in predicting where it does not wait for test data to learn.

In this paper, we are focusing on some well-known classification algorithms like decision tree, naïve Bayes, k-nearest neighbor (KNN), gradient boosting trees (GBT), random forest (RF), support vector machine (SVM), and AC algorithms.

2.3 Associative classification

In this paper, we use the same traditional classifiers that were used in previous studies (such as NB, J48, SVM, GBT, and RF), and we propose to use associative classification (AC) algorithms (such as CBA, CBA2, CMAR, CPAR). On the other hand, we experiment with our approach ACRM as an associative classification algorithm, which depends on two main interesting measures (IMs), confidence and conviction, to extract the class association rules (CARs).

AC uses the rules extracted from frequent patterns to classify the unknown example into a specific class label. Thus, we are looking for the rules that have a strong relationship between the frequent items and the class label. Associative classification for reviews mining (ACRM) integrates association rule discovery and classification to build a classifier for prediction. Based on literature, AC algorithms extract competitive classifiers compared with traditional classifiers (Thabtah 2007).

AC and ACRM algorithms employ the following to improve its accuracy and efficiency over traditional classifiers:

- (1) Construction approach: The AC algorithms build their classifiers from class association rules by extracting all frequent patterns using Frequent Pattern Growth (FP-Growth), which is used to produce a dense FP-Tree. All frequent patterns are represented in the FP-Tree, which is smaller in size than the original dataset. Also, the construction of the FP-Tree uses only two scans for the dataset. Then, the rules are extracted using a pattern growth method. Only a subset of rules is selected based on the occurrences of the class values in the right hand side of the rule. This mechanism does not exist in the traditional classifiers, because the AC algorithms do not need to extract all rules. The memory utilization requires less space in extraction frequent patterns than constructions of traditional classifiers due to FP-Tree compact representation.
- (2) Output rules: the output of AC algorithms is in if-then rules format. The rules are simple and easy for the users to interpret rather than traditional classifier outputs. The updating of the output rules does not require rebuilding the classifier from beginning which happens in NB, J48, SVM, GBT, and RF. Also, all extracted rules are above or equal minimum confidence, which contributes in gaining a high accuracy in the predication. Also, AC algorithms avoid generating redundant rules.

2.3.1 CBA algorithm

Classification based on association rules (CBA) algorithm was proposed in (Bing et al. 1998). CBA depends on the highest confidence rule to classify the new tuples where the first rule

satisfying the tuple is used to classify it. Bing et al. proposed CBA-RG algorithm to generate all CARs with rule pruning capabilities (Bing et al. 1998). CBA-RG uses the same procedures of the Apriori algorithm. The only difference is that CBA-RG increments the support counts of the set of items (X) of the CAR and the CAR rule separately, while the Apriori algorithm updates only one count.

After rule generation, CBA builds its classifier from CARs, which is generated by CBA-RG. CBA algorithm chooses rules with high precedence from all training datasets. A rule has higher “ X ” precedence than “ Y ” if:

- 1- $\text{Conf}(X) > b$,
- 2- $\text{Conf}(X) = \text{conf}(Y)$, but $\text{sup}(X) > \text{sup}(Y)$,
- 3- $\text{Conf}(X) = \text{conf}(Y)$, $\text{sup}(X) = \text{sup}(Y)$, but X generated before Y .

where $\text{Conf}(X)$ is the confidence value of item X , and $\text{Sup}(X)$ is the support value of item X .

CBA applies three main procedures:

- Procedure 1: Sort the generated CARs based on the precedence order.
- Procedure 2: Selecting ordered rules for the classifier by selecting those rules, which cover examples.
- Procedure 3: Discard those rules in the classifier that do not improve the accuracy of the classifier, keep the rule with the lowest error rate, and discard others in the sequence.

2.3.2 CBA2 algorithm

CBA2 considers a new approach proposed in (Liu et al. 2001), where they improve the CBA algorithm into CBA2. The previous CBA algorithm uses single minimum support in the CBA RuleGenerator function, while CBA2 uses multiple minsup values according to class distribution in the training dataset, other than that, both of them work similarly.

The value of each class (c_i) minimum support (minsup_i) is calculated from the frequency distribution for each class ($\text{freqDistr}(c_i)$) and the total minsup (t_minsup) which is given by the user as shown in Eq. 1:

$$\text{minsup}_i = t_minsup \times \text{freqDistr}(c_i) \quad (1)$$

By using this equation, the class with low frequencies will get a fair number of rules comparing classes with high frequencies.

2.3.3 CMAR algorithm

Another type of associative classification algorithm is classification based on multiple association rules (CMAR) (Li et al. 2001). There is a difference between CBA and CMAR; CBA depends on one rule with the highest confidence to classify the tuple. CMAR uses the weighted chi-square ($\text{Max } \chi^2$) method. CMAR follows a different way of extracting frequent items and building classifiers. It applies the FP-Growth algorithm with a pruning strategy to find the rules which are above minsup and minconf. The classification process in CMAR depends on multiple rules. CMAR consumes less memory size and running time than CBA,

but it is not always accurate than CBA. Both CBA and CMAR are more accurate than the decision tree (Li et al. 2001).

2.3.4 CPAR algorithm

Classification based on predictive association rules (CPAR) was proposed in (Yin and Han 2003) and uses the Laplace Accuracy Method. CPAR was built to take advantage of both associative classification and rule-based classification like C4.5, FOIL, and RIPPER. For example, it generates less number of rules than CBA and more rules than C4.5, because CBA generates rules using a greedy algorithm from the remaining dataset, so this rule is not necessarily the best. CPAR chooses the closest rules to the best rule for each example.

2.4 Data mining in software engineering

Software engineering is the process of maintaining, designing, developing, and testing software applications. It ensures that the software is built systematically, faultlessly, [on schedule](#), and on within budget (Ali 2017). There are several kinds of data available in software engineering, such as graphs, facts, figures, and text. Developers need these data to achieve software engineering goals such as finding and fixing bugs, documentation, mailing lists, cost estimation, and source code (Periasamy and Mishbahulhuda 2017).

Software maintenance considers a crucial part of a software development lifecycle (Brijendra and Shikha 2016). Developers depend on several sources related to software engineering maintenance tasks. One of the most important sources of information is user reviews, which help developers in the maintenance phase. User feedback and reviews consider very rich texts with maintenance information. Developers need to know the perspective of the user to see their software concerning software engineering maintenance tasks.

Researchers categorized software maintenance tasks in various ways. According to IEEE international standards (ISO/IEC 14764 2006), software maintenance includes multiple processes as follows:

- (1) Process Implementation
- (2) Problem and Modification Analysis
- (3) Modification Implementation
- (4) Maintenance Review/Acceptance
- (5) Migration
- (6) Retirement

In Problem and Modification Analysis process, software engineers analyze and classify the modification request into two main categories: correction and enhancement. Each one of these types is divided into two other maintenance tasks. Figure 1 demonstrates maintenance tasks according to the IEEE international standards (ISO/IEC 14764 2006).

As we can see from Fig. 1, the correction task is made to fix and solve problems and bugs in the software and can be a preventive procedure when the developer knows the nature of the bugs, or which part of the software holds the problems. Associative rules and classification methods could help the developer to discover a specific pattern for these issues. The enhancement task is either adaptive or perfective. The software needs to be usable and changeable to meet the users' needs. Developers need to know what features the user should request to make

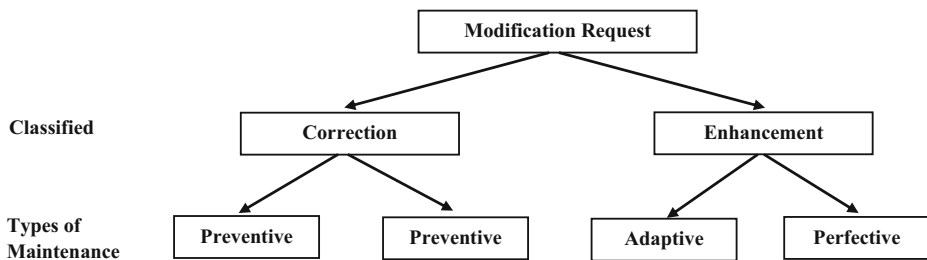


Fig. 1 Maintenance tasks according to IEEE International standards

their software adaptive and perfective. According to these categories of software maintenance tasks, researchers classified user reviews in various ways, but they intend to help developers to understand and perceive user reviews and obtain the benefits of these reviews in the maintenance process. We will present some of these researches and their review classification into maintenance task categories.

Data mining techniques help developers to extract useful information from data available in software engineering to enhance the developing process and software quality (Wang et al. 2017). The data mining tool is very effective for discovering the hidden patterns in SE data, especially for text, where it can help developers to make decisions in any phase of testing or designing their application. Also, they could know what kind of software defects and weaknesses the application contains.

In (Yang and Liang 2015), user reviews were classified into two basic simple categories: functional (FR) and non-functional requirements (NFR). Developers could extract meaningful and practical requirements. Other researches in (Panichella et al. n.d.; Sorbo et al. 2016; Panichella et al. 2016) used four main categories to classify user reviews into software maintenance tasks. These categories are (1) information seeking, where the user wants to get information or assistance from developers or other users, e.g., “I want to know that how to add and delete text and pictures.” (2) Information giving, when users inform the developers about some characteristics of the application, e.g., “It’s simple the desktop app is great too” and “It’s so useful and fast and I just love the dark theme.” (3) Feature request, users in this kind of reviews demand more features like adding a specific button to the interface or ask developers to enhance some options. Usually, they suggest some ideas to enhance the functionality of the software, e.g., “If you add separate Tabs for video and photo we’ll be very happy.” (4) Problem discovery, when users confronted some problems during the phase of installation and updating. Also, users may discover bugs and issues while using the app, e.g., “crashing issue after the update i cant access this app anymore.”

Maalej and Guzman classified user reviews into three categories: feedback about a feature, feature request, and bug report (Guzman and Maalej 2014). Other researchers added a new task called rating to include the reviews that reflect the rating of the app as words in the text. In (Maalej et al. 2016; Maalej and Nabil 2015), reviews are classified into four categories: feature request, bug report, user experience, and rating. Reviews that are related to the user experience category include the user’s opinion about the app. It expresses whether the review is negative or positive and may include their feelings about the app.

From previous researches, we can conclude the following: the developer needs to keep monitoring his software to provide a suitable maintenance process and necessary to consider

user reviews as the primary source of maintenance information. In this paper, we adopted two types of researchers' opinions concerning review classification. The first type is proposed in (Panichella et al. n.d.; Sorbo et al. 2016; Panichella et al. 2016) and the second type is proposed in (Maalej et al. 2016; Maalej and Nabil 2015), and we used two datasets representing these two types.

2.5 Text classification preprocessing

A user review is a short text written by the user in his way, so the text classification reviews need some preprocessing to convert the unstructured data to structured data. In this section, we will clarify text classification preprocessing techniques that are used to prepare the dataset for the classification process.

Text preprocessing is a fundamental part of NLP and very necessary for text mining. Reviews are unstructured data, incomplete, missing, and not organized. Many steps can be applied to the text before classification; these steps should be done in the correct sequence to produce the final input dataset into the classifier. Three main preprocessing steps used in text classification are the following: tokenization, stop word removal, and stemming. All these preprocessing steps aim to produce all words that exist in reviews as a matrix of word vectors where every row represents an independent review. After creating the text word vectors, we can use many classification algorithms (Vijayan et al. 2017).

Word vectors contain the term frequency-inverse document frequency (tf-idf) score (Gurusamy and Kannan 2014), which reflects the importance of the word in the document between groups of multiple documents. tf-idf is calculated as shown in Eq. 2:

$$tf-idf_{(t,d,D)} = tf_{(t,d)} * idf_{(t,D)} \quad (2)$$

$t(t,d)$ is a term frequency, which means how many times a term t has occurred in document d . $idf(t,D)$ (inverse document frequency) measures how the term t is significant in all documents D . It is calculated by dividing the total number of documents N by the number of documents containing the term (df_t), then taking the logarithm of the result, as shown in Eq. 3:

$$idf_{(t,D)} = \log \frac{N}{df_t} \quad (3)$$

The term which has a higher tf-idf in the document is the rarest and strongest term that indicates the document d among all documents D . When a term is repeated in all documents, it becomes less important as an indicator to a particular document. In this case, it will have a lower tf-idf weight. Therefore, using the measured tf-idf provides the following benefits:

- Stop words have less influence on document classification since it occurs in most documents.
- Extracting the words that have a strong indication of the document.
- Building a structured text from unstructured text, which helps researchers to use word vectors of documents.

The first phase of preprocessing is tokenization, which is considered as a significant process in lexical analysis. In this process, a sentence is divided into a sequence of individual words where tokenization uses white spaces and punctuation marks to distinguish the words

(Gurusamy and Kannan 2014). Then, we apply the stop words removal technique where all common words that are meaningless are removed. These stop words are used to join sentences or words. Also, the ones do not contribute to the document subject are removed, like “and,” “the,” “so,” and “always.” Keeping the stop words will cause a lot of noise and an inaccurate classification process in most cases (Ghag and Shah 2015).

The stemming phase is an essential part of preprocessing to find all these derivations that belong to the same word. If stemming was not applied, word vectors will build a new column for every derivation which reduces the frequency of the word and increases the dimension of the dataset. For example, if a document has three derivations of “request” word like “requesting,” “requested,” and “requesting,” then word vectors will contain a column for each one of these words. When stemming is applied, a new column of word vectors will be created for “request” words including all its derivation. Many algorithms are used to apply stem, like Table Lookup Approach, Successor Variety, N-Gram stemmers, and Affix Removal Stemmers (Gurusamy and Kannan 2014). Stop words removal and stemming are important to reduce the dimension of the dataset, and to get rid of data noises. In some text preprocessing, Bigram extraction is applied, where every two sequenced words can be taken as one feature. Bigram increases the dimension of the dataset (Ghag and Shah 2015).

3 Literature review

This section aims to give a general understanding of literature that studied the reviews to help software engineers to understand users’ intent and evaluate their apps according to user reviews.

Many researchers have analyzed app reviews to help developers discover maintenance tasks and facilitate their job. Some researchers are interested in analyzing users’ opinions and users’ emotions. Other studies are focused on reviews such as normal text and they used some text classification methods but with different preprocessing techniques. Every app has feedback from users, but not all reviews are useful for software engineers. It is very important to recognize these reviews as the authors in (Yang and Liang 2015) did. The authors in (Yang and Liang 2015) have proposed a new approach to classify user reviews into functional and non-functional reviews in respect to requirement information. They used NLP techniques and extracted tf-idf values for words to build a regular expression as a classifier.

Ankit and Sunil in (Ankit and Sunil 2017) have presented a review paper about data mining tools and techniques that can be used in software engineering areas. They tried to figure out the software engineering areas, where the data mining techniques can be used. According to Ankit and Sunil, several data sources can be mined using multiple mining techniques such as documentation, source code, bug databases, mailing lists, and software repositories. Then, they summarized the tools according to different categories, such as newly created tools, developed prototypes, traditional data mining tools, implemented, and scripting tools. Also, they associated these tools with software engineering data. For example, classification can be used for documentation data, while clustering, classification, and text retrieval can be used for source code data.

Other authors studied the emotions in the reviews to discover the correlation between emotion and software nature (Martens and Johann 2017; Williams and Mahmoud 2017a; Li et al. 2017). Martens et al. studied the emotional sentiment in user reviews and how useful for software engineers (Martens and Johann 2017). They found a weak correlation between

sentiments and user ratings, but when reviews are classified to maintenance tasks, the sentiment becomes more influential and increases the classification accuracy. In (Williams and Mahmoud 2017a), 1000 tweets were collected from software systems and were used to classify the reviews into three categories of negative sentiment (bug report, frustrated with update, and unsatisfied with update), and three categories for the positive sentiment (satisfaction, anticipation, and excitement). NB, SVM, and SentiStrength were used to classify the reviews. NB and SVM got a higher accuracy than SentiStrength. Li et al. studied the main topics that are of interest to users like performance, battery, stability, usability, memory, price, and security (Li et al. 2017). They created a dataset from 900 reviews that were extracted from the Google play market and identified the topics of comparative reviews using words' tf-idf values and some NLP techniques.

Many researchers studied review classification with respect to maintenance tasks. Different maintenance tasks were used as a class label. Guzman et al. have classified user reviews into seven categories (bug report, feature strength, feature shortcoming, user request, praise, complaint, usage, and scenario) (Guzman and El-Haliby 2015). They used the ensemble of four classifiers (NB, Logistic Regression, Neural Networks, and SVM). They found that neural networks have achieved the best classification accuracy with a precision of 74%, recall of 59%, and F-measure of 64%. Williams and Mahmoud used NB and SVM to classify the reviews into bugs, feature requests, and others (Williams and Mahmoud 2017b). They used the words' tf-idf values in the classification process. Their dataset was extracted from tweets of three applications (Minecraft, WhatsApp, and Snapchat). Ciurumelea et al. (Ciurumelea et al. 2018) have built a tool that classifies reviews based on pre-specified categories and provide evidence of complaints in each category. The percentage of complaints helps the developers of the app to prioritize the complaints of users. The authors in (Palomba et al. 2015; Palomba et al. 2018) have studied the accommodation of user requests that are extracted from the reviews of 100 Android apps. The work found that the developers accommodate the requests and the ratings of apps increased as a result. The authors have proposed a tractability approach from code changes to reviews. This tracking can be utilized to support release planning. In addition, the authors have provided a tool, CRISTAL, which supports the findings of the research. The tool helps development teams manage the changes that increase user satisfaction. Bakiu and Guzman (Bakiu and Guzman 2017) have focused their research on finding user reviews that give information about the usability and user experience of apps. The authors extracted features from user reviews and then applied sentiment analysis and mapped the discovered features to sentiments. The mapping helps in finding the user satisfaction of features in the app. Villarroel et al. (Villarroel et al. 2016) built a tool to classify user reviews into informative (e.g., bug, feature) and non-informative categories. The tool also clusters related reviews for easy inclusion in the next release of the app. The authors used random forest trees to classify user reviews and the DBSCAN for clustering. The data consists of 1000 reviews from 200 Android apps. In Panichella et al. (Panichella et al. n.d.), the authors have proposed a classification of app reviews into maintenance and evolution categories using NLP, text analysis, and sentiment analysis. The combination of these methods showed better results than an individual technique in the classification of user reviews in five classifiers, J48, SVM, logistic regression, naïve Bayes, and AdTree. The methodology was conducted on seven Apple store and Google play apps. Zhou et al. (Zhou et al. 2020) have proposed an approach to classify, cluster, and link software reviews to app integration. The approach allocates user reviews into clusters of similar user reviews.

Authors in (Panichella et al. [n.d.](#); Sorbo et al. [2016](#)) have presented four main categories for review classification: information giving, information seeking, bug reports, and feature requests. They used NLP, text analysis, and sentiment analysis to prepare a dataset for classification. Sorbo et al. in (Sorbo et al. [2016](#)) proposed the SURF approach, which classifies reviews into their related topics besides the maintenance tasks. Many classifiers were used as in (Panichella et al. [n.d.](#)) such as NB, SVM, logistic regression, J48, and AdTree. The authors have conducted many experiments with variations in the training dataset and preprocessing techniques. The experiments have resulted in different precision, recall, and F-measure values. J48 was the best classifier with precision and recall of 75%, and 74%, respectively (using NLP, text analysis, and sentiment analysis). They sampled 20% of the dataset as a training set. Table 1 shows the classification results of the classifiers used in (Panichella et al. [n.d.](#)) when they used all the features that were extracted by NLP, sentiment analysis, and text analysis. While Table 2 shows the classification results for each maintenance task category obtained by J48. We can notice that app users have a strong linguistic pattern when they write a review about a bug or a problem because problem discovery has the highest recall. On the other hand, feature request has a low F-measure value, which indicates that users ask for new features using several ways, which makes it difficult to discover the linguistic pattern in these reviews.

Authors such as in (Maalej et al. [2016](#); Guzman and Maalej [2014](#); Maalej and Nabil [2015](#)) have classified user reviews into the following categories: bug report (or problem discovery), feature request, user experience, and rating. They worked on the same dataset. Decision tree, NB, and maximum entropy (MaxEnt) were used in (Maalej et al. [2016](#); Maalej and Nabil [2015](#)). They used the NB classifier with different NLP techniques (bag of words (BOW), stop words removal, lemmatization, star rating, tenses, and sentiment). Some experiments have combined two or more of the previous techniques to find the best preprocessing technique with the NB. They split the dataset into 70:30, i.e., 70% of the data was allocated for training set with ten cross-validations. They applied two different classification methods: binary classification and multiclass classification as shown in Table 3.

Table 3 shows that the authors in (Maalej et al. [2016](#)) have achieved good results with binary classification. The reviews were classified into two classes (e.g., feature request, not feature request). Whereas for the multiclass, the results were very poor because of the imbalance in the dataset. The rating reviews are about 67% of the dataset. The average F-measure value for the NB, decision tree, and MaxEnt were 0.53, 0.54, and 0.12, respectively. In our study, we use the multiclass classification method after balancing the dataset, since we study all types of reviews together.

In conclusion, review classification is different from one researcher to another in respect to software maintenance tasks. Different techniques can be used for dataset preparation like NLP techniques, sentiments, and text analysis. According to previous works, the best classifiers that are used in review classification are NB and J48. In our study, we are using NB and J48 in all

Table 1 Classification results of Panichella et al. in (Panichella et al. [n.d.](#))

Classifiers	Precision	Recall	F-measure
NB	0.69	0.68	0.65
SVM	0.67	0.68	0.66
Logistic regression	0.45	0.42	0.43
J48	0.75	0.74	0.72
AdTree	0.79	0.72	0.67

Table 2 Results by category for the J48 algorithm from (Panichella et al. [n.d.](#))

Classifiers	Precision	Recall	F-measure
Feature request	0.70	0.22	0.34
Problem discovery	0.87	0.77	0.82
Information seeking	0.71	0.68	0.70
Information giving	0.68	0.90	0.78
Weighted average	0.75	0.74	0.72

classification experiments. In addition, we will use associative classification algorithms to classify user reviews into maintenance tasks. We will use two datasets in our experiments; the first one was reported in (Panichella et al. [n.d.](#)) and the second dataset was reported in (Maalej et al. [2016](#)). In the next section, we will discuss both datasets in more detail.

4 Dataset preparation and methodology

4.1 Dataset preparation

In this paper, we used two datasets in our experiments. We chose these two datasets because many researchers have used them with several classification algorithms and they are available. Also, they contain several categories of software maintenance tasks that are suitable for multiclass classification, and these categories are different in both datasets. This section will show where these datasets are collected from and what dataset preprocessing phases are applied. Moreover, this section highlights the problems and challenges encountered during the dataset preparation.

4.1.1 Dataset collection and description

The first dataset was taken from (Panichella et al. [n.d.](#)) and collected by Panichella et al. We obtained this dataset from Dr. Sebastiano Panichella via email. This dataset contains reviews of the Angry Birds, Dropbox, and Evernote app, which were taken from Apple's App Store; other reviews were taken from Android's Google Play store such as Tripadvisor, PicsArt, Pinterest, and WhatsApp. They created their truth dataset with 1390 reviews from all previously mentioned apps. They classified the reviews into four classes related to the software maintenance task as shown in Table 4. We indicate to this dataset in this paper as "Pan Dataset."

Table 3 F-measures of the classifiers used by Maalej et al. in (Maalej et al. [2016](#))

Classifiers	Classification method	Bug report	Feature request	User experiment	Rating	Avg.
NB	Binary	0.79	0.71	0.81	0.83	0.79
	Multiclass	0.62	0.42	0.5	0.58	0.53
Decision tree	Binary	0.73	0.68	0.78	0.78	0.72
	Multiclass	0.62	0.47	0.53	0.54	0.54
Maximum entropy	Binary	0.66	0.65	0.6	0.69	0.65
	Multiclass	0.14	0.00	0.29	0.04	0.12

Table 4 Pan dataset reviews classes

Tasks (class)	Review number
Feature request	192
Problem discovery	494
Information giving	603
Information seeking	101
Total	1390

This dataset comes as an Excel file, which consists of two attributes. The first one represents the texts of the reviews, and the second attributes represent the class label as one of the maintenance tasks categories. Also, there is another file related to this dataset, which contains the linguistic patterns extracted from these reviews.

The second dataset is used in (Maalej et al. 2016) and prepared by Maalej et al. It is available at the Hamburg University website, a direct link for this dataset can be found on (<https://mast.informatik.uni-hamburg.de/app-review-analysis>). The truth dataset contains 3691 reviews from different Google’s app store and Apple’s app store. We indicate this dataset in the paper as the “maalej dataset.” Table 5 shows the classes of these reviews.

This dataset exists as an Excel file as well. But it has several attributes that represent different data about the reviews. The first attribute represents the texts of the reviews, where each review has the following attributes: review tasks (class) as a text, star rating score, and the number of the past tenses, future, and present as a numerical data. Also, the sentiment score of each review from one to five, and the number of words for each review as numerical data.

4.1.2 Dataset preprocessing

Using different text preprocessing techniques will create a different dataset, where different words will form the final shape of a structured dataset. We applied different sequential preprocessing steps for each dataset according to (Panichella et al. n.d.; Maalej et al. 2016), and we follow the same preprocessing steps that applied in the previous works, to get the same classification results for the classifier that were applied in these previous studies and compare it with the performance of other classifiers as well. The two datasets have reported different feature selection techniques, and therefore, we report all feature selections in (Panichella et al. n.d.; Maalej et al. 2016) to repeat the work and to test the use of other algorithms in addition to the ones previously reported in (Panichella et al. n.d.; Maalej et al. 2016).

Pan dataset Three main preprocessing steps were applied to this dataset as applied in (Panichella et al. n.d.):

Table 5 Maalej dataset review classes

Tasks (class)	Review number
FR (feature request)	252
BR/PD (bug report/problem discovery)	370
UE (user experience)	607
RT (rating)	2461
total	3691

- Text analysis (TA): In this phase, we applied stop word removal (using the English standard stop word list), stemming using English Snowball Stemmer, tokenization, and then weighting the resulting words by calculating tf-idf value for each word. The resulting of this phase is a weighted word vector.
- Natural language processing (NLP): When users write their reviews, usually they follow duplicate linguistic patterns. For example, when a user asks for a new feature, he says: “you should add a more options for font types” or “please, it would be better if you make it black.” These two sentences have different syntax, but these two syntaxes may be repeated from other users when they ask for new features or options. Therefore, it is very important to discover and recognize the syntax of the sentence to get to know the user’s intent. We can notice the following from the first sentence: “you” is subject, “should” is the auxiliary of the main verb, “add” is the main verb for user intent, and “font types” is a feature that user desires. We applied a mapping process between these linguistic patterns and the reviews to find the linguistic syntax for each review if it existed. The resulting of this phase is the existential linguistic pattern in each review.
- Sentiment analysis (SA): Three sentiment analysis degrees were used: positive, negative, and neutral. Sentiment analysis employs an important approach in the maintenance requests. The author intentions can be exploited to help developers in discovering various types of informative reviews. The user frequently includes negative words in the review to expose to a problem related to the app, while using neutral words in the review refers to asking for new features. In (Williams and Mahmoud 2017c), they used sentiment analysis to classify the reviews into three categories of a negative sentiment (bug report, frustrated with update, and unsatisfied with update) and other three categories for a positive sentiment (satisfaction, anticipation, and excitement). In our work, RT reviews indicate to how much users love or hate the app, so the following rules have appeared: (good), (app, love), (awesome), (best), (better), (easy), (excellent). Some rules have a future verb with “great,” “good,” and “love.” Other rules have specific sentiment score like (future_range1, sentiScore_range8), (pastt_range1, sentiScore_range9), where scores 8 and 9 have highly positive sentiment.

We combined these three preprocessing techniques to produce the final structured dataset, which is ready for the classification process. Figure 2 depicts the preprocessing steps on reviews.

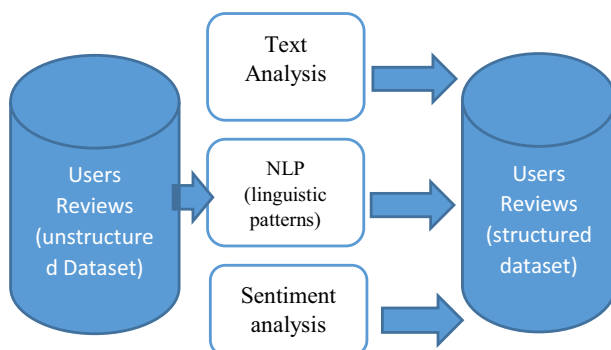


Fig. 2 Preprocessing procedures on Pan dataset

Maalej dataset We applied the same preprocessing steps that were applied in (Maalej et al. 2016), as follows:

- Natural language processing: This phase includes the following techniques: stop word removal, stemming, and bigrams. tf-idf is calculated for the resulting words to produce a word vectors matrix.
- Metadata: which contains the following features:
 - (1) Star rating from 1 to 5 which is given by the user.
 - (2) The number of tenses in each review, past, present, and future. The past tense in user reviews is used for feature description or reporting. On the other hand, the future tense is used for solving problems or suggests some case assumptions. We assume that tenses that are used in reviews reflect in somehow the user intent.
 - (3) Sentiment strength: negative strength from -5 to -1 and positive strength from 1 to 5 are also included in the dataset.

We applied the previous preprocessing steps on the Maalej dataset and combined all extracted features into one dataset to be ready for the classification process.

4.2 Methodology

Our methodology is shown in Fig. 3; we apply different preprocessing techniques on both datasets (Pan and Maalej dataset) to convert the reviews into a structured dataset. Then, we train different classification algorithms using the KEEL software (Triguero et al. 2017) and the RapidMiner studio (Arunadevi et al. 2018; Mans et al. 2014). We start with feature extraction using some NLP techniques, and we use some traditional algorithms besides AC algorithms for classification processes.

4.2.1 Feature selection

In the preprocessed phase, we extract all words and other features (sentiments score and bigrams). The produced dataset contains poor features and effective features for the classification process. For instance, some words exist only in one or two reviews. These words have less influence from the words with five to ten frequency. The word vectors contain all the words of the reviews, and that could produce a huge size of the dataset with thousands of words depending on the length and the number of reviews.

We will apply the feature selection technique in some of our experiments using information gain (IG) and chi-square. Several researchers used IG for features ranking and selection like (Ding and Fu 2018; Zdravevski et al. 2015; Alhaj et al. 2016; Pratiwi and Adiwijaya 2018; Shen et al. 2017). To measure the relevance of attribute X in class Y , we apply Eq. 4. (Pratiwi and Adiwijaya 2018)

$$IG = H(Y) - H(Y/X) \quad (4)$$

where $H(Y)$ is calculated by Eq. 5. It represents the entropy of the class and $H(Y/X)$ is the conditional entropy of class given attribute (Pratiwi and Adiwijaya 2018).

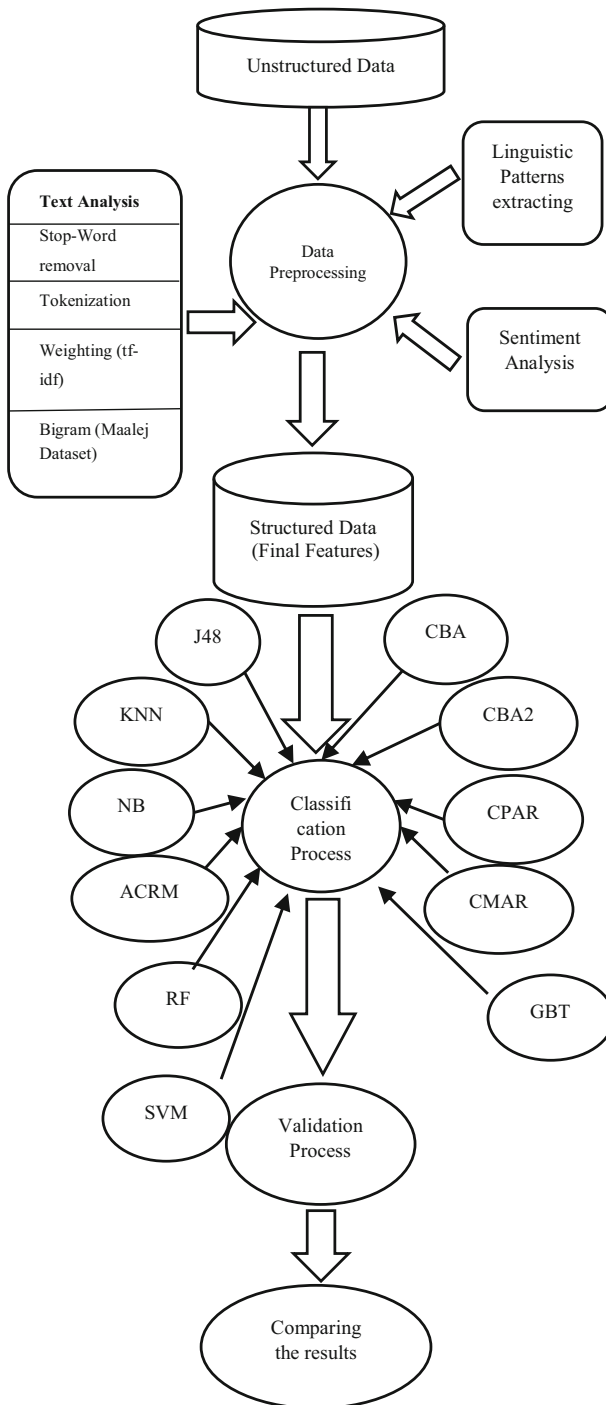


Fig. 3 The methodology followed in this study

$$H(Y) = \sum_{c \in C} -P(Y)_i \log_2 P(Y)_i \quad (5)$$

Chi-square measures the correlation between the feature f_k and the class c_i as shown in Eq. 6. (Vora and Yang 2017)

$$\text{Chi-square } (f_k, c_i) = \frac{N(XD - CY)^2}{(X + C)(Y + D)(X + Y)(C + D)} \quad (6)$$

where N is the total number of reviews. X is the number of reviews in class c_i that contains the feature f_k . B is the number of reviews that contains the feature f_k in other classes. C is the number of reviews in class c_i that do not contain feature f_k . D is the number of reviews that do not contain the feature f_k in other classes. Each feature has a score for each class as shown in Eq. 3, then all features are ranked according to max (chi-square (f_k, c_i)).

4.2.2 The proposed approach: associative classification for reviews mining

We designed our proposed approach ACRM using RapidMiner. In this section, we will discuss the approach's workflow and the differences between ACRM and other associative classifications. Figure 4 shows the workflow.

The following sub-processes represent ACRM approach:

- (1) **FP-Growth:** After data preprocessing, we get a binomial dataset, then we apply the FP-Growth algorithm on this dataset. The output of FP-Growth represents the frequent itemsets of the training data, which will be used to generate the rules in the next step. We should specify the minimum support value for the FP-Growth algorithm.

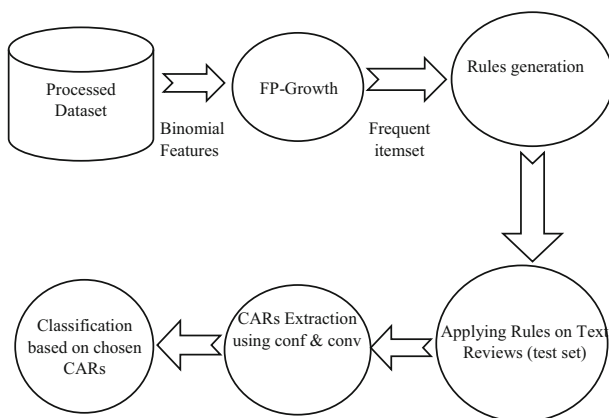


Fig. 4 ACRM workflow

- (2) Rules generation from the training dataset: In this step, we generate the rules according to a specific confidence value from all the frequent itemsets.
- (3) Applying rules to reviews text: In this sub-process, the extracted rules are tested against the items that belong to a specific review from the test set. This process is considered as a mapping between the rule and review items. For instance, the $X, Y \rightarrow Z$ means if items (words) in the LHS of the rule exist in reviews items, then the rule satisfies the review. Our approach depends on two IMs to extract the rules. We chose the rules with maximum confidence and maximum conviction values separately that satisfy each review. The rule with high confidence is not necessary has the highest conviction value and that leads to producing two different groups of rules according to confidence and conviction values.
- (4) CARs extraction: In this process, we extract the rules that satisfy the following form ($X, Y, \dots, Z \rightarrow \text{class label}$) while the remaining are filtered out.

We tried many experiments for IMs as we will explain in section five. We found that using confidence as a first measure then conviction gives us the best CARs for the classification process. Therefore, confidence is more intuitive than the conviction in the case of generating rules from reviews items. If we have two rules for a different class label and they have the same confidence value, this will cause misleading in the classification process. However, if we have other IM, we can use it to determine the class label of that review. The following pseudocode represents this sub-process:

For each review

For each group of rules with the same class label that satisfy the review.

Find rule with maximum conf and rule with maximum conviction.

End for

End for

- (5) Classification Process using extracted CARs: We build the classifier from CARs, which is calculated from the confidence measure where CAR with maximum confidence classifies the rule to its class label; if two CARs have the same value, we use conviction value.

Comparing AC algorithms with ACRM Table 6 illustrates the major differences between AC algorithms in respect to interesting measures (IMs) and the algorithms that are used to produce frequent itemsets.

Table 6 AC algorithm comparison

AC algorithms	Frequent items	IMs
ACRM	FP-Growth	Confidence, conviction
CBA	Apriori	Support, confidence (Bing et al. 1998)
CBA2	Apriori	Support, confidence (Liu et al. 2001)
CMAR	FP-Growth	Chi-square test (χ^2) (Li et al. 2001)
CPAR	Generates rules directly, using FOIL algorithm	Laplace (Yin and Han 2003)

ACRM has the following specifications:

- (a) It uses the FP-Growth algorithm, which is more efficient and less time consuming than the Apriori algorithm (Dharmaraajan and Dorairangaswamy 2016).
- (b) It uses confidence and conviction to build the classifier, but it gives the priority to confidence to adopt the CARs for the classification process.

5 Experiments and results

The purpose of this study is to find the best classifiers to classify user reviews into software maintenance tasks and to propose the ACRM approach. We conduct several experiments with different classifiers.

5.1 Choosing minimum support value

Minimum support is a parameter that is used to generate the frequent itemsets. When the minsup value is too low, we will get a huge number of meaningless frequent itemsets and that leads to building many CARs. On the other hand, low minsup requires a long execution time. If the minsup value is too high, few rules will satisfy the review (Bai et al. 2018). From the former idea, choosing minimum support value depends on the nature of the frequency of the items. In this study, reviews are short text, which usually contain a specific number of words. Therefore, the probability for repeating a specific word in the same review or other reviews is not usually high. The minimum support value for reviews dataset should be lower than news articles and documents, which have more repeated words.

To know the proper minsup value for the reviews dataset, we run the ACRM approach with Pan dataset using different minsup values. For each value, we apply ten cross-validations. From Fig. 5, we notice that if we increase the minsup above 0.0125, the accuracy will go down. If we decrease minsup under 0.007, the number of frequent items will decrease and the accuracy will go down as well. Generally, the minimum support value that is suitable for the review text is very low. The best minsup = [0.007–0.0125] because the frequencies of the important terms are usually low in app reviews. Therefore, we use minsup = 0.01 with all associative classification algorithms used in the experiments.

5.2 Choosing K value of the KNN algorithm

KNN algorithms need to specify K value. We apply KNN several times on Pan dataset $K = (1-30)$, with ten cross-validations for each K value. Figure 6 shows the classification accuracy of KNN with multiple K values. When the K value is bigger than 14, the accuracy decreases. In contrast, the best value is 11 when KNN gives the best accuracy. It is extremely fundamental to find the best K value because the KNN algorithm searches the training set to find the k closest examples to the unclassified example, and then it classifies the unknown example by a majority vote of the closest neighbors' class label. We use the Euclidian distance to calculate the similarity between the new example and its neighbors. Hence, we are using $K = 11$ in all our experiments.

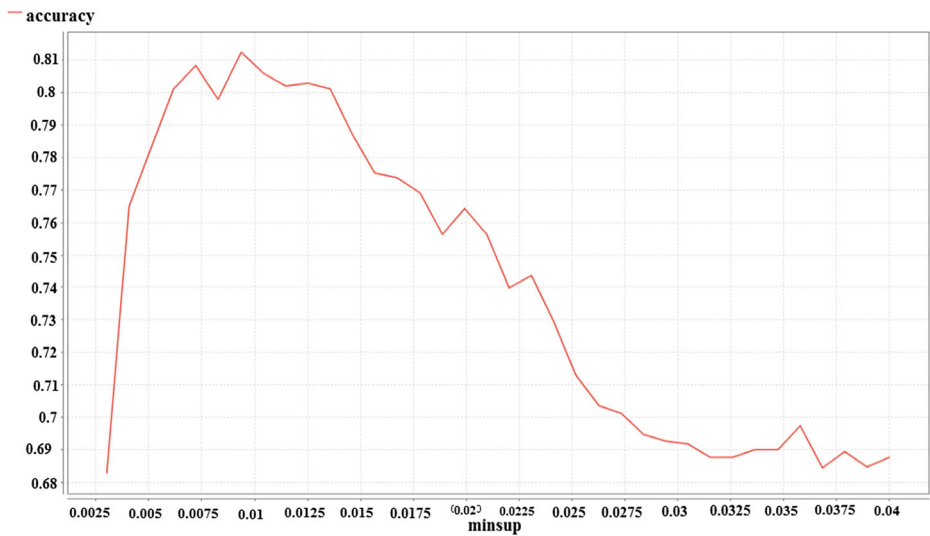


Fig. 5 Accuracy based on multiple minsup values

5.3 What are the best interesting measures used in ACRM approach

This experiment aims to find the most suitable interesting measures (IMs) for reviews' classification that will be used in our approach. We need two IMs to run ACRM, so we try different IMs (confidence, conviction, Laplace, lift) on Pan dataset as shown in Table 7. We notice that there are some results close to each other. Overall, the best results were when we use confidence and conviction. Using conf-conviction couple has the best F-score and accuracy (0.771, 0.791), respectively. Hence, in all our experiments, we use confidence and conviction for review classification.

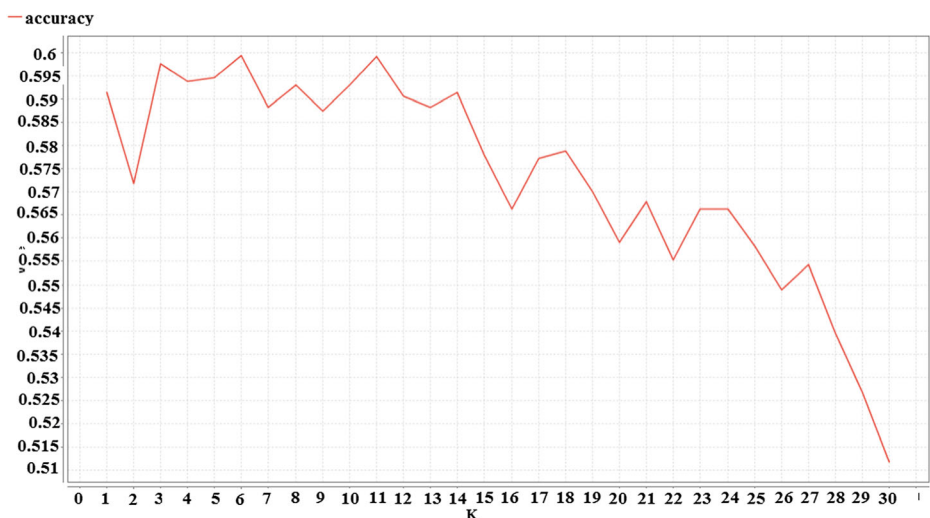


Fig. 6 KNN accuracy based on multiple k values

Table 7 ACRM with multiple IMs

IMs	Precision	Recall	F-score	Accuracy
Conf-conviction	0.806	0.745	0.774	0.791
Conf-Laplace	0.806	0.741	0.772	0.788
Conf-lift	0.805	0.742	0.772	0.792
Conviction-conf	0.784	0.771	0.777	0.788
Conviction-Laplace	0.800	0.750	0.774	0.788
Conviction-lift	0.799	0.751	0.774	0.789
Lift-conf	0.646	0.630	0.638	0.657
Laplace-conf	0.810	0.736	0.771	0.791
Laplace-conviction	0.803	0.741	0.772	0.793
Laplace-lift	0.803	0.741	0.771	0.795

5.4 PAN dataset

5.4.1 Classification with all features

Table 8 presents the classification results using precision, recall for each software maintenance task. In addition, we use the overall precision, recall, F-score, and accuracy of all classifiers used in this experiment. The preprocessing phase produces 1900 features.

We notice that NB, KNN, and RF have the lowest overall F-score. NB gives a good result when the dependency degree is high between the elements. In this experiment, NB was used for review classification and reviews have a low dependency between its words. In addition, important words that belong to the same type of reviews have a low frequency as well, so NB gave low F-score results. While KNN depends on the closest neighbors for classification, it faces difficulty in similarity calculation between the items of the reviews, where most tf-idf values for most words are low and the dimension of the dataset is big.

CPAR, ACRM, and GBT have the highest F-score value of all. CPAR has the highest F-score with 0.795, while the ACRM approach has the highest accuracy with 0.791. Figure 7 shows the average performance of all classifiers. We notice that the associative classifiers have a better performance with respect to precision. ACRM and CPAR generate strong rules that discover the hidden patterns between the items, and these rules were more influential in the

Table 8 Classification of reviews using all features (Pan dataset)

Classifier	FR		IGv		IS		PD		Mean		F-score	ACC
	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec		
J48	0.70	0.67	0.77	0.79	0.71	0.72	0.83	0.81	0.75	0.75	0.75	0.77
NB	0.46	0.61	0.64	0.55	0.85	0.17	0.58	0.71	0.63	0.51	0.56	0.58
KNN	0.41	0.19	0.58	0.74	0.57	0.45	0.66	0.62	0.56	0.50	0.53	0.59
ACRM	0.70	0.64	0.82	0.77	0.92	0.66	0.78	0.92	0.81	0.75	0.78	0.79
CBA	0.76	0.58	0.74	0.83	0.83	0.24	0.80	0.90	0.78	0.64	0.70	0.77
CBA 2	0.69	0.74	0.80	0.79	0.61	0.33	0.81	0.88	0.73	0.68	0.70	0.78
CPAR	0.77	0.58	0.80	0.89	0.81	0.78	0.95	0.60	0.81	0.78	0.80	0.77
CMAR	0.92	0.36	0.57	0.96	0.50	0.03	0.95	0.60	0.73	0.49	0.59	0.67
SVM	0.84	0.53	0.70	0.85	0.71	0.32	0.78	0.79	0.76	0.62	0.68	0.74
GBT	0.82	0.55	0.72	0.90	0.85	0.74	0.90	0.78	0.82	0.74	0.78	0.79
RF	0.25	0.01	0.44	0.99	0.25	0.01	0.91	0.11	0.46	0.28	0.35	0.46

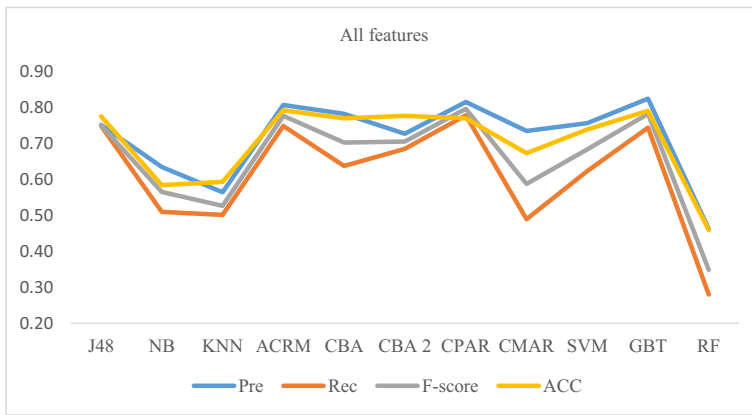


Fig. 7 Average performance for all classifiers (Pan dataset)

classification process than the rules generated by J48. J48 has higher precision, recall, F-score, and accuracy than CMAR, KNN, and NB. CBA and CMAR have a large difference between precision and recall. GBT has the highest precision value of all with 0.833, then come CPAR and ACRM, which have precision value with 0.8144 and 0.8057, respectively. GBT has lower recall value than CPAR, ACRM, and ACRM, so it has less ability to mark the true reviews into its correct class. RF was the worse classifier in this experiment.

- **FR:** Feature request reviews have the highest precision with CMAR (0.92), but it has a low recall value (0.36). This means that among the reviews that are labeled as feature requests, 92% of them are true feature requests. But all reviews that are truly feature request, only 36% of them are labeled as feature requests. So CMAR has less ability to predict the actual feature requests. We have five classifiers that have precision above 0.75 and they are CBA, CMAR, CPAR, SVM, and GBT. In addition, four classifiers have a recall value above 0.60 and they are ACRM, J48, NB, and CPAR. The other classifiers have a weak classification of the true FR reviews as FR reviews. In general, there is an imbalance between precision value and the recall value for all classifiers. Recall values for all classifiers except CBA2 were very low; this means that users can ask developers for new features in several different patterns, making it difficult to recognize the common patterns. Also, this result can be explained by the low number of feature requests in the dataset.
- **IGv:** Information giving reviews have the highest precision with ACRM (0.822), where it has a good recall value with 0.774. ACRM has a good balance between the precision and recall values, which makes it a decent classifier to predict IGv reviews. While CMAR has the highest recall with 0.964, but it has a low precision value with 0.568. RF has the lowest value of all.
- **IS:** Information seeking reviews have the highest precision with ACRM (0.91). The next highest precision is for GBT with 0.85 and NB with 0.85 as well. CPAR has the highest recall value with 0.78. GBT and J48 come after with 0.74 and 0.72, respectively, while the rest have recall under 0.50.
- **PD:** problem discovery (bugs report) reviews have the highest precision with CMAR (0.9466), but CMAR has the lowest recall with 0.60. ACRM has the highest recall with

0.918; this means from all reviews that are truly PD reviews, about 92% of them were labeled as PD reviews. The CARs generated by ACRM could discover the recurrent patterns that users are following when they write a review about a bug or a problem.

In conclusion, we notice from the previous analysis that ACRM, CPAR, and GBT are the best classifiers for this experiment, where ACRM has the highest accuracy with 0.791, and GBT has the highest precision. The worst classifier was FR, NB, and KNN.

We conduct this experiment with the same preprocessing steps that were applied in (Panichella et al. n.d.). We combined the features from text analysis (stop word removal and stemming), sentiment analysis, and NLP (linguistic patterns). We will compare the results of this experiment with the results of the best classifiers in (Panichella et al. n.d.), i.e., J48. Table 9 shows the precision and recall values for J48 in (Panichella et al. n.d.), where it has used different sizes of the training set (20%, 40%, and 60%) with ten cross-validations, and combining the three text preprocessing techniques (text analysis, sentiment, and NLP). The performance results in our experiment with J48 are relatively close to the ones with the same preprocessing steps (pre = 0.75, rec = 0.748). ACRM and CPAR achieve better performance than J48.

5.4.2 Classification using feature selection

In this experiment, we discuss the impact of the feature selection techniques on reviews' classification. We applied feature selection to keep the strong features that carry influence on the classification process. We applied two feature selection techniques, we used information gain (IG) and chi-square. We extracted the top 10% features after applying feature selection.

Feature selection using information gain Feature selection aims to observe classifiers' behavior after removing inefficient words and features. Table 10 shows the classification results when applying the IG feature selection. We notice that CPAR, ACRM, and GBT have the highest F-score with 0.799, 0.785, and 0.766, where the accuracy was 0.802, 0.797, and 0.796 respectively. ACRM, CPAR, and GBT have close F-score and accuracy values, but they have different precision and recall at classes' level. They have the highest values even after the feature selection process. J48 comes next with F-score (0.759) and accuracy (0.777). CBA, CBA2, CMAR, and RF have high precision and low recall (same behavior even without feature selection). AC algorithms have the best precision among the classifiers, which means they have a strong ability to predict the correct class.

From Table 8, we see that NB and KNN have noticeable improvements. For instance, NB F-score was 0.564 when all the features existed; F-score rises to 0.718 with feature selection, because feature selection removes the weak features that are independent of each other; and this serves the way how the NB algorithm works. Feature selection with the use of IG also improves the performance of the precision value for all classifiers. This means using feature selection

Table 9 Performance results of J48 in the related work

Training set size	Pre	Rec
J48-20	0.752	0.742
J48-40	0.743	0.721
J48-60	0.743	0.721

Table 10 Classification results using IG (Pan dataset)

Classifier	FR		IGv		IS		PD		Mean		F-score	ACC
	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec		
J48	0.69	0.66	0.77	0.79	0.78	0.74	0.83	0.83	0.76	0.75	0.76	0.78
NB	0.81	0.79	0.58	0.84	0.65	0.58	0.86	0.65	0.72	0.71	0.72	0.72
KNN	0.82	0.30	0.62	0.90	0.62	0.69	0.81	0.59	0.72	0.62	0.66	0.68
ACRM	0.70	0.66	0.84	0.78	0.91	0.68	0.79	0.92	0.81	0.76	0.79	0.80
CBA	0.76	0.59	0.73	0.83	0.92	0.23	0.81	0.89	0.80	0.64	0.71	0.77
CBA 2	0.74	0.77	0.79	0.80	0.85	0.29	0.81	0.90	0.80	0.69	0.74	0.79
CPAR	0.81	0.59	0.78	0.89	0.83	0.77	0.89	0.86	0.83	0.77	0.80	0.80
CMAR	0.87	0.52	0.63	0.95	0.86	0.31	0.94	0.66	0.82	0.61	0.70	0.73
SVM	0.84	0.51	0.64	0.96	0.75	0.40	0.93	0.64	0.79	0.62	0.70	0.73
GBT	0.83	0.46	0.67	0.94	0.92	0.71	0.93	0.72	0.84	0.71	0.77	0.80
RF	0.97	0.19	0.54	0.98	0.88	0.21	0.94	0.52	0.83	0.48	0.61	0.64

reduces false-positive reviews. RF has an interesting improvement with precision value after the feature selection was applied, where it grows from 0.46 to 0.83. Associative classification algorithms are less affected by noise in the dataset; words and features that are not related to reviews do not influence AC algorithms, especially, ACRM and CPAR. Associative classification algorithms depend on the most influential rules to build the classifiers while RF, KNN, and NB, depend on all features in the classification process, so it is necessary to apply feature selection techniques when we use them in the reviews' classification process.

Feature selection using chi-square Table 11 represents the classification results for all classifiers when we use the chi-square as a feature selection technique. CPAR and ACRM have the highest F-score with 0.787 and 0.777 and accuracy with 0.787 and 0.788, respectively. GBT comes next with F-score (0.767) and accuracy (0.77).

Using IG and chi-square enhances the classification performance, but IG gives a higher performance in most cases, as it is shown in Fig. 8. The median value of the F-score after applying the chi-square selection is higher and shows better values.

Overall, feature selection improves the performance of classifiers, as shown in Fig. 8, where most of the inefficient features that are considered as outliers were removed from the dataset. Feature selection shows improvements on the median of the F-score as shown in the boxplots. To confirm the findings, we have conducted a Wilcoxon signed-rank test to find the significance and the effect (Cliff's delta) of the improvements when feature selection is used to improve the prediction models (Cliff 1993). Table 12 shows the comparison between two models, before and after the feature selection technique. The significance values (i.e., p value) that are less than 0.05 show that the differences are statistically significant. Therefore, from the results in Table 12, we notice significant improvements when feature selection is used on software review classification. Cliff's delta shows the effect of the differences. The minus values are showing the direction in favor of feature selection over using all features. The delta values that are different from 0 show a large effect. We can notice that the IG feature selection has more effect than the chi-square method.

We can summarize our findings from the experiments on the Pan dataset as follows:

- The F-score and the accuracy of software maintenance tasks are not dominant for a particular task; some classifiers give good classification for specific kinds of reviews, but it is not necessary to be the best one for others.

Table 11 Classification results when using chi-square (Pan dataset)

Classifier	FR		IGv		IS		PD		Mean		F-score	ACC
	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec		
J48	0.6719	0.6898	0.7809	0.7708	0.74	0.7551	0.8235	0.8235	0.7541	0.7598	0.7569	0.776
NB	0.6569	0.6979	0.7541	0.603	0.5294	0.63	0.6564	0.7692	0.6492	0.675	0.6619	0.6764
KNN	0.8293	0.3542	0.6029	0.9326	0.8679	0.46	0.8371	0.5814	0.7843	0.582	0.6682	0.6853
ACRM	0.6633	0.6806	0.8455	0.749	0.8846	0.69	0.7853	0.9227	0.7947	0.7606	0.7773	0.7878
CBA	0.7838	0.6042	0.7425	0.8315	0.8519	0.23	0.804	0.9005	0.7955	0.6415	0.7103	0.7738
CBA_2	0.6943	0.6979	0.7835	0.7996	0.8286	0.29	0.8061	0.9027	0.7781	0.6726	0.7215	0.7801
CPAR	0.75	0.5714	0.7614	0.8671	0.8652	0.7857	0.8684	0.8326	0.8113	0.7642	0.787	0.7872
CMAR	0.8462	0.5156	0.6311	0.9419	0.825	0.33	0.9331	0.6629	0.8088	0.6126	0.6972	0.732
SVM	0.780	0.427	0.600	0.97	0.886	0.39	0.956	0.547	0.806	0.584	0.6776	0.69
GBT	0.8272	0.4739	0.678	0.93	0.922	0.71	0.919	0.723	0.8367	0.709	0.7679	0.77
RF	0.9333	0.145	0.537	0.971	0.952	0.2	0.944	0.5384	0.8419	0.4640	0.5983	0.6348

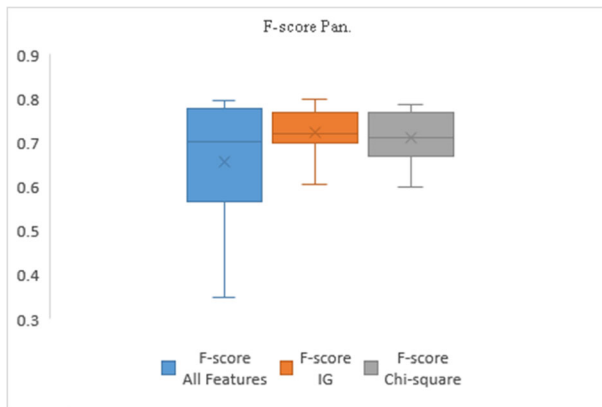


Fig. 8 Classifier performance before and after feature selection

- AC algorithms have a better average performance than J48, KNN, RF, SVM, and NB.
- ACRM, CPAR, and GBT have the highest performance in all the experiments.
- ACRM has the strongest balancing between the precision and recall values.
- Feature selection improves the performance of the classifiers, where NB and KNN have a significant improvement. Associative classification algorithms have resulted in a little improvement because they are less affected by inefficient features, and they depend on the strongest rules in the classification process.
- Feature selection using IG improves the precision value for all classifiers.
- IG feature selection is better than chi-square for reviews' classification.

5.4.3 Rules for review classification

We have found the following rules of importance to users:

- IGv reviews express users' opinions about some app characteristics. Some couple of words, when coming together within a review, indicate to be an IGv review (LHS of the CARs) like (need, find), (recommend, high), (idea, great), and (need, when). So these sets of words are considered to be the LHS of the rules alongside IGv as the RHS of the rule.
- FR reviews have strong words that are related to improving the app and request features, such as the pairs (would, able), (please, add), (would, better), (add, option), (app, feature), (app, wish), and (improve). We notice that these words express user intent for feature request; for instance, when a user uses "wish" thus mean he has a request for an option.

Table 12 The Wilcoxon signed-rank tests and the Cliff's delta values before and after IG feature selection

	F-score		Accuracy	
	All-IG	All-ch_sq	All-IG	All-ch_sq
Z	-2.49	-1.87	-1.60	-2.58
p value (2-tailed)	0.01	0.06	0.11	0.01
Cliff's delta	-0.27	-0.16	-0.26	-0.13

- PD reviews usually indicate something wrong that is occurring about using the app. The strongest CARs have words and couples like (crash) (crash, app), (fix), (please fix) (update, when), (problem), (issue), (bug). We notice when “please” come with “fix” indicate to PD, but if it comes with “add” indicate to FR. When a word like (load) comes with PD rules, this means many of PD reviews have loading issues.
- IS reviews usually appear with question words, like (why, what, how), because user intent is obtaining information and clarifications about some features or characteristics. In addition, it has a syntax pattern like ([something] results|app).

5.5 Maalej dataset

The Maalej dataset has a big difference between the numbers of classes, as shown in Table 5. We have 2461 out of 3691 reviews that belong to rating reviews which represent around 67% of the dataset. The rest of the dataset represents the reviews that belong to three classes. Rating reviews are the least important reviews to study because these reviews demonstrate the user’s love or hate for the review. The minority classes (FR, PD, and UE) are the most demanding. Maalej et al.’s experiments depend on the binary classification to analyze reviews because the multiclass classification grants poor results, and that returns to the majority class, which is a rating class (Maalej et al. 2016).

In our study, we use this dataset to find the performance of the classifiers with a variety of software maintenance tasks using multiclass classification. We will balance the dataset according to the minority class, which is the feature request class. We will take 252 reviews from each class randomly and create a balanced dataset for multiclass classification. Then, we can compare the algorithms according to review multiclass classifications.

In this section, we discuss the impact of the bigram feature as a preprocessing step. Hence, we apply three experiments, one without including bigram features in the preprocessing phase and the other two experiments with bigram features (the top 10% and 20% of the selected features). Since using the bigram feature produces a large number of features (every two-sequence words as one feature), we need to apply feature selection to reduce the noise in the data.

5.5.1 Classification without bigram features and feature selection

In this experiment, we exclude the bigram feature, i.e., the word vectors that have single words only. After we applied preprocessing procedures, we get 2008 features for the classification process. Table 13 represents the classification results using precision and recall for each classifier. In addition, we report the precision, recall, F-score, and accuracy of all classifiers.

ACRM has the highest mean precision, recall, F-score, and accuracy (0.7746, 0.773, 0.7737, 0.7729), respectively. In other words, ACRM has the least number of false-positive and false-negative reviews. Thus, 77.4% of all reviews of a predicted specific label are classified correctly and 77.3% of the true reviews are classified into its correct class. Also, it generates rules that are more influential in the classification process than the rules generated from other AC algorithms. CPAR has F-score close to ACRM with 0.7706 but with low accuracy (0.724). RF comes the second with an accuracy value of 0.744 and comes third with precision value. KNN, NB, SVM, and J48 have the lowest performance results as shown in

Table 13 Classification results of reviews using all features (Maalej dataset)

Classifier	PD		RT		FR		UE		Mean		F-score	ACC
	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec	Pre	Rec		
J48	0.5714	0.7778	0.7395	0.631	0.7771	0.5119	0.6972	0.7857	0.6963	0.6766	0.6863	0.6767
NB	0.5083	0.6071	0.5806	0.5	0.4786	0.4881	0.6781	0.627	0.5614	0.5556	0.5585	0.5555
KNN	0.4025	0.377	0.6033	0.5794	0.3333	0.3294	0.5267	0.5873	0.4665	0.4683	0.4674	0.4683
ACRM	0.7778	0.7778	0.8297	0.754	0.7611	0.6825	0.795	0.877	0.7746	0.7728	0.7737	0.7729
CBA	0.6643	0.754	0.7051	0.6548	0.7054	0.6746	0.7247	0.7103	0.6999	0.6984	0.6991	0.6988
CBA_2	0.6572	0.7381	0.7033	0.6865	0.6818	0.6548	0.7511	0.7063	0.6983	0.6964	0.6973	0.6968
CPAR	0.7066	0.737	0.8053	0.7811	0.7746	0.6846	0.797	0.8797	0.7709	0.7706	0.7707	0.7241
CMAR	0.6421	0.7805	0.7944	0.6217	0.7205	0.6818	0.7679	0.8018	0.7313	0.7215	0.7264	0.6769
SVM	0.7038	0.5753	0.5141	0.7936	0.619	0.464	0.776	0.6904	0.65346	0.6309	0.642	0.6309
GBT	0.638	0.7976	0.6422	0.833	0.8106	0.5436	0.898	0.7023	0.7473	0.7192	0.733	0.7193
RF	0.7218	0.7619	0.7716	0.6706	0.6474	0.71428	0.853	0.8293	0.7485	0.74404	0.7462	0.7441

Fig. 9. CMAR has a higher F-score value than CBA and CBA2, while CBA and CBA2 have higher accuracy than CMAR.

The results of the classification in the four classes of reviews are provided as follows:

- PD: ACRM has the best precision with 0.7778, while GBT has the best recall with 0.796 but low precision with 0.638, then ACRM and J48 come next with 0.7778. ACRM is the best classifier to detect the problem discovery reviews, where precision and recall are balanced and high.
- RT: ACRM has the highest precision value with 0.8297, and it has a recall value of 0.754. GBT has the highest recall value with 0.833, and it has a precision value of 0.642. This means ACRM has the least number of false-positive RT reviews, and GBT has the least number of false-negative reviews. ACRM is the best classifier to detect the problem discovery reviews, where precision and recall are balanced and high.
- FR: GBT has the highest precision value with 0.81, but low recall value with 0.543; so GBT has the least number of false-positive FR reviews. RF has the highest recall with 0.741. CPAR, ACRM, and CMAR come next recall values (0.6846, 0.6825, and 0.681) respectively.
- UE: CPAR and ACRM have the highest recall value with 0.877 and 0.879, respectively, and they have precision values of 0.795 and 0.797. Thus, CPAR and ACRM are the best to predict the actual UE reviews. GBT and RF have the highest precision value with 0.898 and 0.853; thus, GBT and RF have the least number of false-positive FR reviews.

In conclusion, we notice from the previous analysis that ACRM has the strongest balancing between the precision and recall values, which makes it get the highest main of precision, recall, F-score, and accuracy. The worst classifier was NB and KNN in this experiment.

5.5.2 Classification with bigram features

We get 10,122 features when we extract bigram features from all reviews in the preprocessing phase. This huge number of features includes many inefficient features.

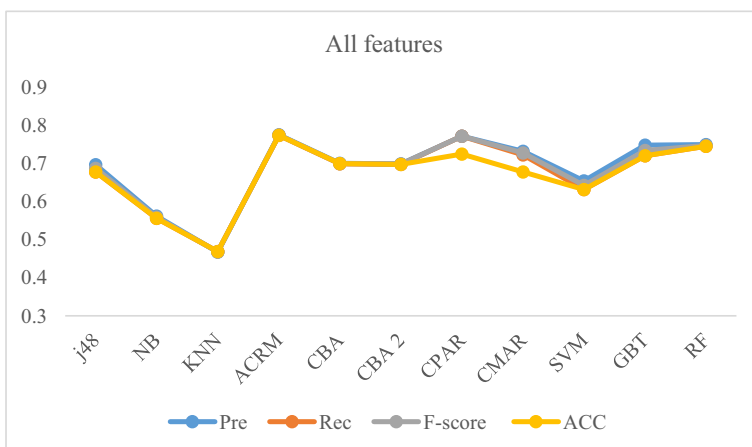


Fig. 9 The average performance for all classifiers (Maalej dataset)

In the preprocessing phase, all words and features from text analysis are included in the word vectors matrix, even the inefficient words that have a frequency equal to one or two in all reviews. We apply two experiments on this dataset with the extraction of bigram features, where we use feature selection based on IG with using the top 10% features and the top 20% features. Tables 14 and 15 represent the classification performance for all classifiers according to precision, recall, F-score, and accuracy with the top 20% and top 10% features. We exclude the CMAR algorithm since it takes a very long execution time compared with all other classifiers.

ACRM, NB, and RF have the best accuracy and F-score values with (0.762, 0.7699, 0.766), (0.7643, 0.7629, 0.766), respectively. NB has the highest accuracy when we are using bigram features in the processed dataset with the top 20%. The performance of NB has improved noticeably. For instance, the F-score value has increased from 0.5585 to 0.7699. This is because the linguistic patterns become more obvious when we take every two sequenced words together as one feature, and that increases the dependency between the features, and that increases the ability of NB to be more accurate when calculating the probability of dependency between the features. KNN is still the worst, and there are no improvements. CPAR has the highest F-score (0.7773) but with low accuracy (0.7164).

ACRM, RF, and NB have the highest precision, recall, F-score, and accuracy. ACRM is considered the best classifier when we add bigram features with the top 10%. At the same time, NB shows improvement from 0.5585 to 0.7678. KNN is still the worst, and there are no improvements.

To conclude all classifiers' performance, we present a boxplot of the performance of the classifiers according to F-score in three cases (without bigram feature, with bigram (top 20%), and with bigram (top 10%) as shown in Fig. 10). The median values for each set of classifiers show better results with bigram features (20% and 10%). The distribution of the classifier values is very wide without bigram. The classifiers are more consistent when used with bigram features than otherwise. To confirm the findings, we conducted a Wilcoxon signed-rank test to find the significance and the effect (Cliff's delta) of the improvements when bigram features are used to improve the prediction models. Table 16 shows the comparisons between two models, with and without a bigram feature. The significance values that are less than 0.05 show that the differences are statistically significant. Therefore, we notice significant improvements when feature selection is used on software review

Table 14 Classification results for the top 20% of the selected features using bigram features

Classifier	Precision	Recall	F-score	ACC
J48	0.7051	0.6875	0.6962	0.6875
NB	0.7777	0.7698	0.7737	0.7699
KNN	0.4813	0.4802	0.4807	0.4802
ACRM	0.7641	0.7645	0.7643	0.7629
CBA	0.7239	0.7173	0.7206	0.7181
CBA 2	0.7072	0.7014	0.7043	0.7018
CPAR	0.7771	0.7776	0.7773	0.7164
SVM	0.7779	0.745	0.761	0.745
GBT	0.7583	0.73	0.744	0.73
RF	0.7665	0.7668	0.7666	0.7669

Table 15 Classification results for the top 10% of the selected features using bigram features

Classifier	Precision	Recall	F-score	ACC
J48	0.6937	0.6825	0.6881	0.6824
NB	0.7686	0.7679	0.7682	0.7678
KNN	0.4612	0.4633	0.4622	0.4643
ACRM	0.7823	0.7815	0.7819	0.7808
CBA	0.7225	0.7192	0.7208	0.7193
CBA 2	0.7128	0.7113	0.712	0.7113
CPAR	0.7575	0.7586	0.758	0.7115
SVM	0.7499	0.7192	0.734	0.7192
GBT	0.7591	0.729	0.743	0.6824
RF	0.775	0.7728	0.7739	0.7728

classification. Cliff's delta shows the effect of the differences. The minus values are showing the direction in favor of feature selection over using all features. The delta values are different from 0 and show a large effect. Therefore, feature selection has effect in improving classifiers.

We notice that most of the classifiers have stable behavior with little improvement after using bigram feature, except NB. NB has the worst performance without bigram feature; then, it shows a better performance using bigram feature because bigram feature has a strong influence on the dependency between the features, i.e., every two sequenced words are connected as one feature. In general, ACRM, CPAR, and RF have a high performance in all three cases.

We can summarize the results of the experiments to Maalej dataset as follows:

- ACRM has the highest mean precision, recall, F-score, and accuracy, without using bigram features and feature selection.
- The classification performance of the NB is highly increased with bigram features.
- ACRM, NB, CPAR, and RF have the highest F-score values when using bigram features.
- Best F-score and accuracy values were with bigram features when using ACRM with the top 10% of the selected features, where F-score = 0.7819 and accuracy = 0.7808.

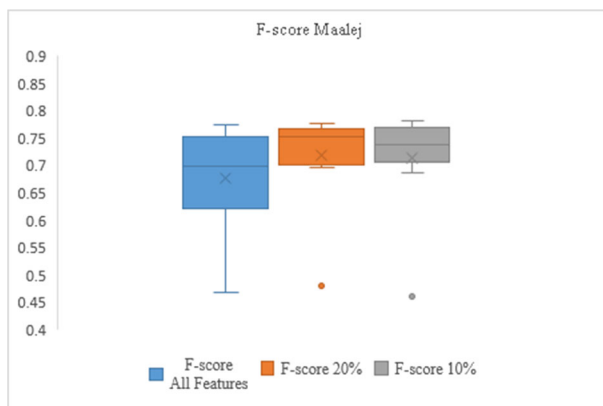
**Fig. 10** Comparing classifiers with bigram feature with top 20% and top 10% of the selected features

Table 16 The Wilcoxon signed-rank tests and Cliff's delta values with and without bigram features

	F-score		Accuracy	
	All-20%	All-10%	All-20%	
Z	−2.49	−2.09	−2.29	−2.29
p value (2-tailed)	0.01	0.04	0.02	0.02
Cliff's delta	−0.35	−0.32	−0.34	−0.21

5.5.3 Rules for review classification

We have found the following rules of importance to users:

- PD reviews have words indicating issues happening in the app, like (app, crash), (app, time, crash), (app, updat, fix), (fix, problem). We use star rating and tenses in preprocessed data, so we get the following LHS from CARs as well: (ratingg_range1, bug), (ratingg_range1, crash) these two rules mean when words “crash” and “bug” come with a review and star rating = 1 then the review is more likely to be a PD review. In addition, we have (future_range1, fix, download), (future_range1, updat, problem); as rule conclusion, this means when a future verb comes with “update” and “download,” the review is classified as a PD review.
- FR reviews have common words in LHS of the CARs like (feature), (add, option), (improve); some rules indicate that users are politely asking the developer for a feature or an option like (would, nice) (would, love) (please, option). Some CARs related to rating stars, such as (ratingg_range1, feature), which means when star rating = 1 and review has “feature” word, then the review is more likely to be an FR review.
- UE reviews indicate what the user is discovering when they are using the app, and it is considered as preventive information for software engineers. We notice the following rules: (future_range1, whi), (future_range1, how), (future_range1, updat), (future_range1, when); all these rules contain a word come with a future verb. Also, we have (video), (voice); these words indicate that users' interest is about the quality or the characteristics of the video and voice.
- RT reviews in Maalej dataset indicate how much users love or hate the app, so the following rules have appeared: (good), (app, love), (awesome), (best), (better), (easy), (excellent). Some rules have a future verb with “great,” “good,” and “love.” Other rules have specific sentiment score like (future_range1, sentiScore_range8), (pastt_range1, sentiScore_range9), where scores 8 and 9 have a highly positive sentiment.

6 Threats to validity

Threats to internal validity concern the truth in datasets under research. The datasets are provided by other researchers. The researchers followed an error-prone human judgement. The reviews were classified into different maintenance tasks by at least two annotators. However, the reviews were validated to assure that both annotators had a similar assignment. The authors in (Panichella et al. [n.d.](#)) calculated the

disagreement in annotations and found it was about 5%. The authors in (Maalej et al. 2016) took several measures to mitigate the internal threats in review classification. The authors created how-to guide to define review types. The annotators should agree on one classification to consider it.

The external validity is about the generalizability of the results. In this research, we replicated previous works in (Panichella et al. n.d.; Maalej et al. 2016) and we added more machine learners to extend the works further. In addition, we conducted feature selection and found improvements on classifiers after the feature selection. However, the machine learning algorithms considered in this research do not cover all possible machine learning such as unsupervised learning. The feature selection techniques do not represent every possible technique and therefore the conclusions are limited to these techniques. The reviews were extracted from large apps that represent different domains of application such as social media, games, businesses, cloud storage systems, and media. The apps come from Google Play Store and Apple Store. These stores cover over 75% of the app market (Maalej et al. 2016). The reviewing style considered in this research may not apply to other stores such as Amazon store. In addition, all reviews considered in this study were written in the English language and more assumptions may be considered when applied to other natural languages.

7 Conclusions and future work

In this paper, an ACRM approach was developed to enable the automatic classification of app reviews into software maintenance tasks. This work required preprocessing techniques using NLP and text analysis to build a structured dataset from app reviews. We used two datasets to apply our experiments. We adopted several preprocessing steps to extract useful features for the classification process as a first stage. Then, as a second stage, we used feature selection techniques such as IG and chi-square to remove inefficient features. As the third stage comes along, we used several classifiers (J48, NB, KNN, and AC algorithms) to the best classifier for classifying app reviews. As the fourth stage, we identified the best CARs to be used to classify the app reviews.

Overall, AC algorithms have a better average performance than J48, KNN, and NB. The best classifiers were ACRM approach and CPAR algorithm in all the experiments that were applied on Pan dataset. Also, ACRM has the strongest balancing between the precision and recall values. Using AC algorithms was very efficient with respect to review classification, where it facilitates rules extraction for developers to specify user intent. Feature selection using IG and chi-square shows significant improvements compared with classifiers using all features. The IG effect size was better than chi-square on review classification. The use of 20% and 10% of bigram also shows significant improvement in review classification compared with classifiers without bigram. In Maalej dataset, the best F-score and accuracy values were for bigram features when using ACRM with the top 10% of the selected features.

As for future work, it is recommended that the study can be expanded by combining more text preprocessing techniques (e.g., using different stemming algorithms). Additionally, we intend to apply other data mining techniques such as clustering to investigate other ways to understand user reviews.

References

- Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules in large databases. VLDB '94. In *Proceedings of the 20th International Conference on Very Large Data Bases* (pp. 487–499). San Jose: IBM Almaden Research Center.
- Agrawal, R., Imieliński, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. *ACM SIGMOD Record*, 22(2), 207–216.
- Alhaj, T. A., Siraj, M. M., Zainal, A., Elshoush, H. T., & Elhaj, F. (2016). Feature selection using information gain for improved structural-based alert correlation. *PLoS One*, 11(11), e0166017.
- Ali, K. (2017). A study of software development life cycle process models. *International Journal of Advanced Research in Computer Science*, 8(1), 15–23.
- Ankit A, Sunil S (2017) A review paper on software engineering areas implementing data mining tools & techniques. *International Journal of Computational Intelligence Research (IJCIR)*. 559-574.
- Arunadevi J, Ramya S, Ramesh Raja M (2018) A study of classification algorithms using Rapidminer, *International Journal of Pure and Applied Mathematics*. 15977-15988.
- Bai, A., Deshpande, P. S., & Dhabu, M. (2018). Selective database projections based approach for mining high-utility Itemsets. *IEEE Access*, 6, 14389–14409.
- Bakiu E. and Guzman E., Which feature is unusable? Detecting usability and user experience issues from user reviews. 2017 IEEE 25th International Requirements Engineering Conference Workshops (REW). Lisbon pp 182-187.
- Bing, L., Wynne, H., & Yiming, M. (1998). Integrating classification and association rule mining. In *Proceedings of the Fourth International Conference on Knowledge Discovery and Data Mining (KDD '98)* (pp. 80–86).
- Brijendra S, Shikha G (2016) The impact of software development process on software quality: a review. 2016 8th International Conference on Computational Intelligence and Communication Networks (CICN), Tehri, pp. 666-672.
- Ciurumelea A, Panichella S, and Gall H. (2018). Automated user reviews analyser. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings (ICSE '18)*. Association for Computing Machinery, New York, NY, USA, 317–318.
- Cliff, N. (1993). Dominance statistics: ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3), 494–509.
- Dharmaraajan K, Dorairangaswamy MA (2016) Analysis of FP-growth and Apriori algorithms on pattern discovery from weblog data. 2016 IEEE International Conference on Advances in Computer Applications (ICACA).
- Ding, J., & Fu, L. (2018). A hybrid feature selection algorithm based on information gain and sequential forward floating search. *Journal of Intelligence Computation*, 9(3), 93.
- Ghag KV, Shah K (2015) Comparative analysis of effect of stopwords removal on sentiment classification. 2015 International Conference on Computer, Communication and Control (IC4).
- Gurusamy V, Kannan S (2014) Preprocessing techniques for text mining. RTRICS.
- Guzman E, El-Haliby M, and Bruegge B (2015) Ensemble methods for app review classification: an approach for software evolution (N). 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), Lincoln, NE, 771–776.
- Guzman E, Maalej W (2014) How do users like this feature? A fine grained sentiment analysis of app reviews. 2014 IEEE 22nd International Requirements Engineering Conference (RE). 153-162.
- Han J, Pei J, and Yin Y (2000) Mining frequent patterns without candidate generation. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data (SIGMOD '00)*. 1-12.
- ISO/IEC 14764:2006 [Internet]. Developing standards. [cited 2018 Nov12]. Available from: <https://www.iso.org/standard/39064.html>.
- Li W, Han J, Pei J (2001) CMAR: accurate and efficient classification based on multiple class-association rules. *Proceedings 2001 IEEE International Conference on Data Mining*, 369-376.
- Li Y, Jia B, Guo Y, Chen X (2017) Mining user reviews for mobile app comparisons. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*. 1(3): 1–15.
- Liu, B., Ma, Y., & Wong, C.-K. (2001). Classification using association rules: weaknesses and enhancements. In *Data Mining for Scientific and Engineering Applications Massive Computing* (pp. 591–605).
- Maalej W, Nabil H (2015) Bug report, feature request, or simply praise? On automatically classifying app reviews. 2015 IEEE 23rd International Requirements Engineering Conference (RE). 116-125.
- Maalej, W., Kurtanović, Z., Nabil, H., & Stanik, C. (2016). On the automatic classification of app reviews. *Requirements Engineering*, 21(3), 311–331.

- Mans, R. S., van der Aalst, W. M. P., & Verbeek, H. M. W. (2014). Supporting process mining workflows with RapidProM. In L. Limonad & B. Weber (Eds.), *BPM Demo Sessions 2014* (pp. 56–60). Eindhoven, September 20, 2014). [CEUR-WS.org](https://ceur-ws.org/): co-located with BPM 2014.
- Martens D, and Johann T (2017) On the emotion of users in app reviews, 2nd International Workshop on Emotion Awareness in Software Engineering (SEmotion), Buenos Aires, 8–14.
- Palomba F, Linares-Vásquez M, Bavota G, Oliveto R, Di Penta M, Shihyanyk D, De Lucia A. (2015) User reviews matter! Tracking crowdsourced reviews to support evolution of successful apps. 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME), Bremen. pp. 291–300.
- Palomba F, Linares-Vásquez M, Bavota G, Oliveto R, Di Penta M, Shihyanyk D, De Lucia A, (2018) Crowdsourcing user reviews to support the evolution of mobile apps, *Journal of Systems and Software*. Volume 137. Pages 143–162. **ISSN 0164-1212**.
- Panichella S, Sorbo AD, Guzman E, Visaggio CA, Canfora G, Gall HC (2016) ARdoc: app reviews development oriented classifier. Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering - FSE 2016. 1023–1027.
- Panichella, S., Di Sorbo, A., Guzman, E., Visaggio, C., Canfora, G., Gall, H., & How Can, I. Improve my app? Classifying user reviews for software maintenance and evolution. In *Proc. of the International Conference on Software Maintenance and Evolution (ICSME)* p. to.
- Periasamy R, Mishbahulhuda A (2017) Applications of data mining techniques in software engineering. *International Journal of Advanced Research in Computer Science and Software Engineering*. 304–307.
- Pratiwi AI, Adiwijaya (2018) On the feature selection and classification based on information gain for document sentiment analysis. *Applied Computational Intelligence and Soft Computing*. 1–5.
- Shen, J., Xia, J., Zhang, X., & Jia, W. (2017). Sliding block-based hybrid feature subset selection in network traffic. *IEEE Access*, 5, 18179–18186.
- Sorbo AD, Panichella S, Alexandru CV, Shimagaki J, Visaggio CA, Canfora G, et al. (2016) What would users change in my app? Summarizing app reviews for recommending software changes. Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering - FSE 2016. 499–510.
- Thabtah F, A review of associative classification mining, *The Knowledge Engineering Review*, Volume 22 , Issue 1 (March 2007),Pages 37–65, 2007.
- Triguero, I., González, S., Moyano, J. M., García, S., Alcalá-Fdez, J., Luengo, J., et al. (2017). KEEL 3.0: an open source software for multi-stage analysis in data mining. *International Journal of Computational Intelligence System*, 10(1), 1238.
- Umadevi S and Marseline K (2017) A survey on data mining classification algorithms, 2017 International Conference on Signal Processing and Communication (ICSPC), Coimbatore, 264–268.
- Vijayan V, Bindu K, Parameswaran L (2017) A comprehensive study of text classification algorithms. 2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI). 1109–1113.
- Villarroel, L., Bavota, G., Russo, B., Oliveto, R., & Di Penta, M. (2016). Release planning of mobile apps based on user reviews. In *Proceedings of the 38th International Conference on Software Engineering (ICSE '16)* (pp. 14–24). New York: Association for Computing Machinery.
- Vora, S., & Yang, H. (2017). A comprehensive study of eleven feature selection algorithms and their impact on text classification. *Computing Conference, 2017*, 440–449.
- Wang H, Bai L, Jiezhang M, Zhang J and Li Q (2017) Software testing data analysis based on data mining. 2017 4th International Conference on Information Science and Control Engineering (ICISCE) 682–687.
- Williams G, Mahmoud A (2017a) Analyzing, classifying, and interpreting emotions in software users' tweets. 2017 IEEE/ACM 2nd International Workshop on Emotion Awareness in Software Engineering (SEmotion). 2–7.
- Williams G, Mahmoud A (2017b) Mining twitter data for a more responsive software engineering process. 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C). 280–282.
- Williams G, Mahmoud A. Analyzing, classifying, and interpreting emotions in software users tweets. 2017 IEEE/ACM 2nd International Workshop on Emotion Awareness in Software Engineering (SEmotion). 2017c; 2–7.
- Yang H, Liang P (2015) Identification and classification of requirements from app user reviews. Proceedings of the 27th International Conference on Software Engineering and Knowledge Engineering.
- Yin X, Han J (2003) CPAR: classification based on predictive association rules. Proceedings of the 2003 SIAM International Conference on Data Mining. 331–336.
- Zdravetski, E., Lameski, P., Kulakov, A., Jakimovski, B., Filiposka, S., & Trajanov, D. (2015). Feature ranking based on information gain for large classification problems with MapReduce. *IEEE Trustcom/BigDataSE/ISPA, 2015*, 186–191.

Zhou, Y., Su, Y., Chen, T., Huang, Z., Gall, H. C., & Panichella, S. (2020). User review-based change file localization for mobile applications. *IEEE Transactions on Software Engineering*, 1.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Assem Radi Al-Hawari obtained his Bachelor's Degree in Computer Science at Mutah University, Karak, Jordan, and hold an M.Sc. in Computer Science at Jordan University of Science and Technology, Irbid, Jordan, 2019. His Master's thesis was "Classification of Application Reviews into Software Engineering's Maintenance Tasks Using Data Mining Techniques". His interests are data mining, text mining, machine learning with python and image processing.



Hassan Najadat is an Associate Professor of Computer Science at Jordan University of Science and Technology. He earned his Ph.D. in Computer Science from North Dakota State University, ND, USA, in 2005. His research interests include analyzing datasets using data mining techniques to develop intelligence applications in text mining, sentiment analysis, health information, accounting information systems, educational data mining, security, and data envelopment analysis. He has published more than 50 papers on data mining including the books "Classification of brain diseases using MRI texture: Decision Tree and Genetic Algorithm" and "Mining Data Envelopment Analysis using Clustering Approach: for Heterogeneous Decision Making Units."



Raed Shatnawi received the M.Sc. degree in software engineering in 2004 and the PhD degree in Computer Science from the University of Alabama in Huntsville. He is currently an associate professor in the Department of Software Engineering at Jordan University of Science and Technology. He has published many papers in high-ranked journals and conferences (IEEE, ScienceDirect, and Interscience). He has reviewed papers for the *Journal of Systems and Software*, *Empirical Software Engineering*, and *Information Sciences*, and many international conferences. His main interests are in software metrics, software refactoring, software maintenance, and open-source systems development.